NASA

NAME S. MUHILAN

SUBJECT Graphics & Multimedia CLASS CSD ROLL NO.

Sr.No.	DATE	TOPIC / PROJECT / ASSIGNMENT	REMARK
1.	7/8/26	Breshnam algorithm - line	
2.	7/8/26	Breshnam algorithm - circle & ellipse	
3.	15/8/26	Midpt algorithm - Rectangle	
4.	15/8/26	Midpt algorithm - ellipse & circle	
5	21/8/26	Shape transformation for Triangle	
6.	21/8/26	Polygon clipping algorithm	
7	28/8/26	DDA Algorithm	
8.	28/8/26	2D - Rectangle transformation	
9.	4/9/26	Smiling face	
10.	4/9/26	Python Multimedia Interface	
11.	8/9/26	Scan conversion & Flood fill algo	
12	11/9/26	Multimedia Messaging receiver app	
13.	11/9/26	3D transformation of Cube	
14.	18/9/26	Sutherland clipping algorithm	
15	18/9/26	3D transformation of Cone	
16.	25/9/26	Rendering a 3D object	
17	25/9/26	Multimedia integration for multimedia	
18	2/10/26	Capture and display video	
19	2/10/26	Object using DDA algorithm	

Completed
21/10/26

RAJALAKSHMI
ENGINEERING COLLEGE

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY Chennai

BONAFIDE CERTIFICATE

NAME S. MUHILAN

ACADEMIC YEAR 2025-26 SEMESTER V BRANCH B.E. C.S.D

UNIVERSITY REGISTER No. 211624104033

Certified that this is the bonafide record of work done by the above student in the IT23E31 Graphics & Multimedia Laboratory during the year 2025 - 2026

30/10/26

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on .....

Internal Examiner External Examiner

Ex.no.:-1
Date:-7/8/26

DRAW LINE USING BRESENHAM'S ALGORITHM

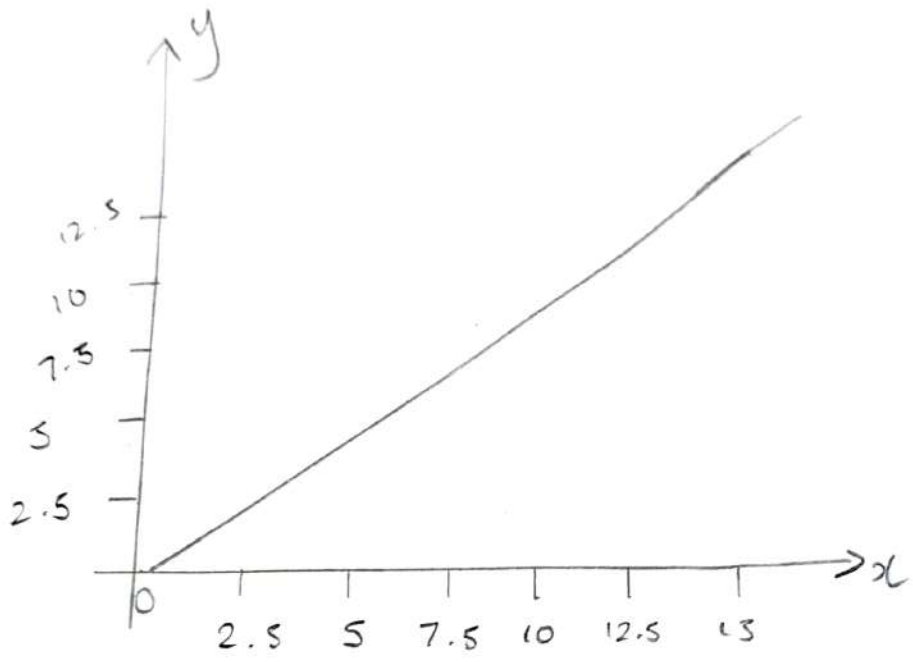
AIM:-

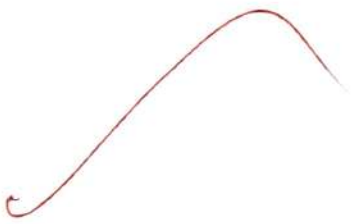
To plot a graph using Bresham's line drawing algorithm.

PROGRAM:-

```
import matplotlib.pyplot as plt
def bresham (x0, y0, xn, yn):
    dx = xn - x0
    dy = yn - y0
    p = 2 * dy - dx
    x = x0
    y = y0
    x_points = [x]
    y_points = [y]
    while x < xn:
        x += 1
        if p < 0:
            p = p + 2 * dy
        else:
            y += 1
```

Output :-





```
        p = p + 2 * dy - 2 * dx

        x_points.append(x)
        y_points.append(y)

    return x_points, y_points

x0 = int(input("Enter x0: "))
y0 = int(input("Enter y0: "))
xn = int(input("Enter xn: "))
yn = int(input("Enter yn: "))

return points

def plot_line(points, color):
    x = [p[0] for p in points]
    y = [p[1] for p in points]

    plt.plot(x, y, color = color, linewidth =
             solid_capstyle = 'round',
             solid_joinstyle = 'round')

def draw_square(points, color):
def draw_square(x, y, side):
    return (.
```

```
bresenham_Line(x+side, y+side,
                x, y+side)
bresham_Line(x, y+side, x, y);]
```

7/8/26

RESULT:-

Hence the square is successfully drawn using bresham's algorithm

Ex.no.:-2
Date:-7/8/26

Draw ~~Line, Square~~, Circle and Ellipse using Bresenham's Algorithm

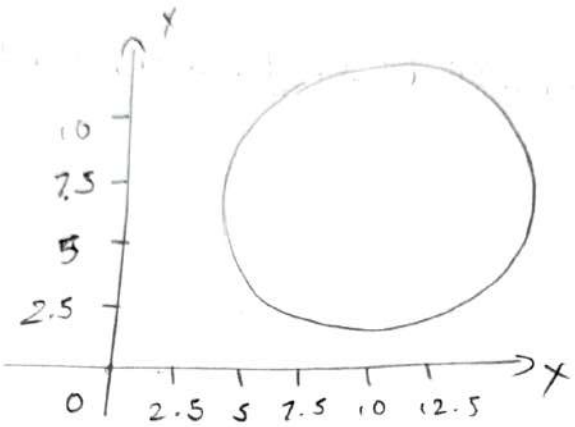
AIM:-

To plot a ~~graph~~ using Brenham's Algorithm to draw ~~line~~, circle, ellipse and ~~square~~

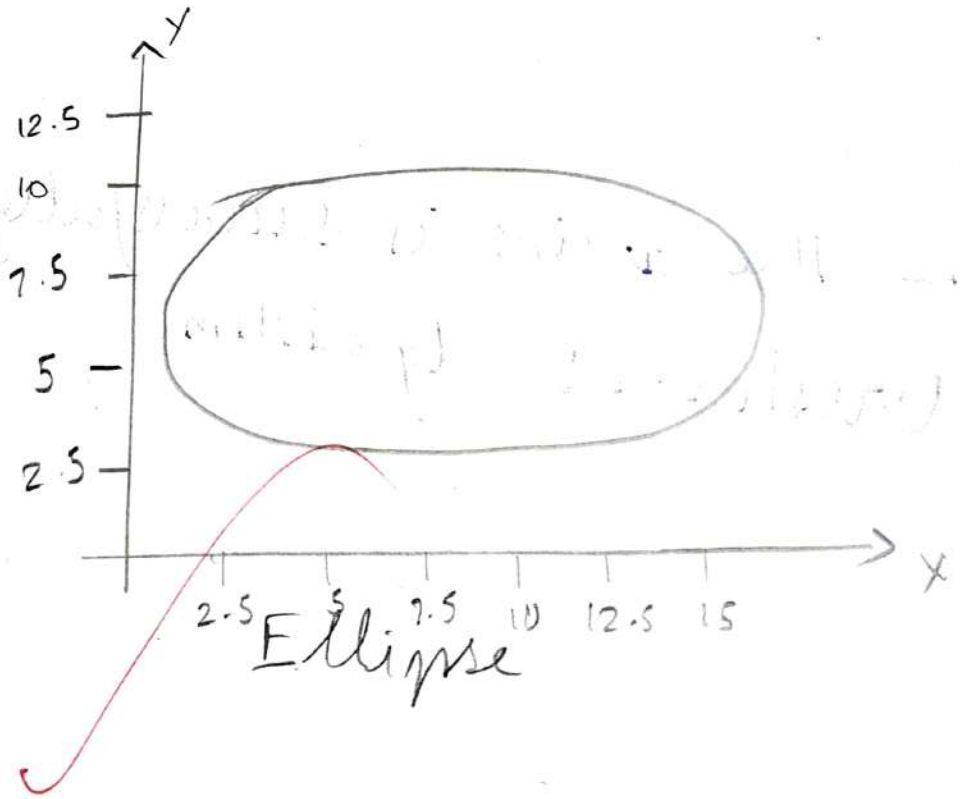
PROGRAM:-

```
import matplotlib.pyplot as plt
~~import numpy as np~~
p = []
def ~~bresenham_line~~ sym (x1, y1, ~~x2, y2~~, e = False):
    points = []
    dx = abs(x2 - x1)
    dy = abs(y2 - y1)
    pts = [(x, y), (-x, y), (x, -y), (-x, -y)]
    p.extend(pts + ([(y, x), (-y, x), (y, -x), (-y, -x)]
        if e else []))
```

Output :-



circle



Ellipse

```
if int(input("1. circle 2. Ellipse : ")) == 1:
    r = 100
    x, y, d = 0, r, 3-2*r
    while x <= y:
        sym(x, y, True)
        if d < 0:
            d += 4*x + 6
        else:
            d += 4*(x-y)+10
            y -= 1
        x += 1
else:
    rx, ry = 120, 80
    x, y = 0, ry
    d = ry*ry
    while 2*ry*ry + rx*rx <= 2*rx*rx*ry:
        sym(x, y)
        if (d < 0):
            d += 2*ry*ry + ry*ry*x + ry*ry
d = ry*ry*(x+0.5)**2 + rx*rx*(y-1)**2
```

```
while y >= 0:
    sym(x,y)
    if d>0:
        d -= 2*rx*rx*y + rx*rx
    else:
        x += 1

x,y = zip(*set(P))
plt.scatter(x,y,s=5,c='b')
plt.axis('equal')
plt.axis("off")
plt.show()
```

RESULT:-

Hence the ellipse and circle have been drawn successfully

Ex. no :- 3
Date :- 15/8/24

RECTANGLE

AIM :- To draw a Rectangle using mid pt algorithm

PROGRAM :-

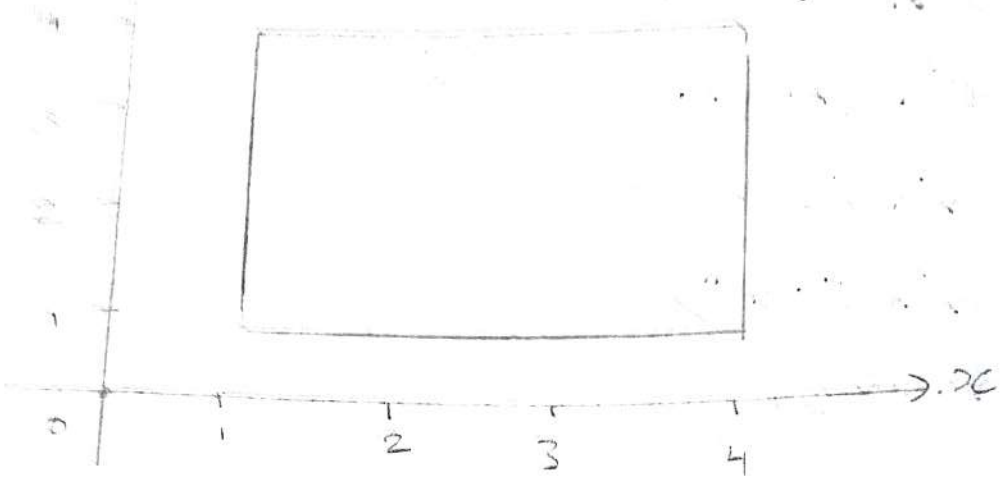
```
import matplotlib.pyplot as plt

def midpoint_line(x1, y1, x2, y2):
    points = []
    dx = x2 - x1
    dy = y2 - y1

    ix = 1
    if dx < 0:
        ix = -1
        dx = -dx

    iy = 1
    if dy < 0:
        iy = -1
        dy = -dy

    x, y = x1, y1
    points.append((x, y))
```



Rectangle

```
if dx > dy:
    d = 2 * dy - dx
    for _ in range(dx):
        if d < 0:
            d += 2 * dy
        else:
            y += iy
            d += 2 * (dy - dx)
        x += ix
        points.append((x, y))
else:
    d = 2 * dx - dy
    for _ in range(dy):
        if dy < 0:
            d += 2 * dx
        else:
            x += ix
            d += 2 * (dx - dy)
        y += iy
        points.append((x, y))
return points
```

x, y = 2, 2

width, height = 0.5

tl = (x, y)

tr = (x + width, y)

br = (x + width, y + height)

bl = (x, y + height)

paint = (

midpaint_line (*tl, *tr)[:-1] +

midpaint_line (*tr, *br)[:-1] +

RESULT:-

Hence The rectangle have been drawn successfully using midpt algorithm

Ex no :-4

Date :-15/8/26

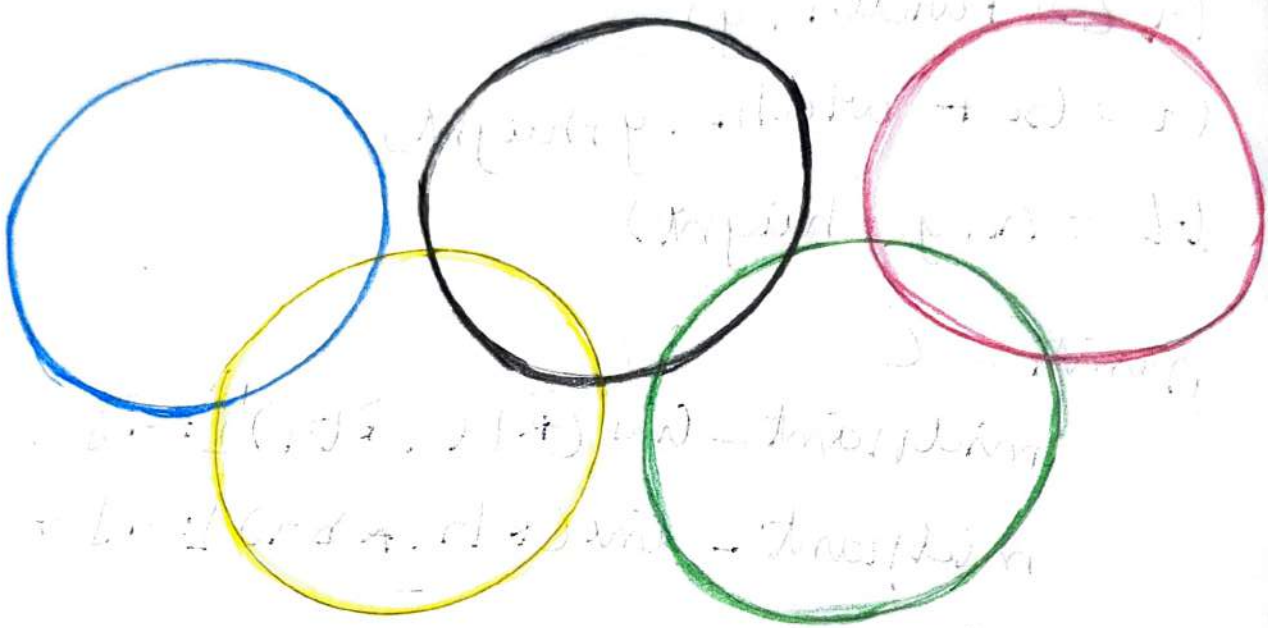
MIDPOINT ALGORITHM FOR OLYMPIC RING

AIM:-

To draw the olympic ring logo using Mid point Algorithm

PROGRAM:-

```
import matplotlib.pyplot as plt
import math
def midpt-ll (rx, ry, xc, yc):
    points = []
    x = 0
    y = ry
    dx = 2 * ry * ry * x
    dy = 2 * rx * rx * y
    p1 = ry * ry - rx * 0.25 * rx * rx
```



```
while dx<dy:
    points.extend([
      (xc+x, yc+y),
      (xc-x, yc+y),
      (xc-x, yc-y),
      (xc+x, yc-y)])

p2 = (ny * ny * (x+0.5)**2 +
      nx * nx * (y-1)**2 - nx*nx*ny*ny)

while y>=0:
    points.extend([
    (xc+x, yc+y),
    (xc-x, yc+y),
    (xc-x, yc-y),
    (xc+x, yc-y)])

points = list(set(points))
points.sort(key = lambda p: math.atan2
                (p[1]-yc, p[0]-xc))

x=[p[0] for p in points]
y=[p[1] for p in points]
```

```
plt.figure(figsize=(12,7), dpi=200)
radius=40
draw_ring(60,90, radius,'blue')
draw_ring(150,90, radius,'black')
draw_ring(240,90, radius,'red')
draw_ring(105,45, radius,'gold')
draw_ring(195,45, radius,'green')
plt.show()
```

RESULT :-

Hence the Olympics ring logo is drawn successfully.

Ex. no.:-5

Date :-21/8/26 Shape Transformation for Triangle

AIM:-

To implement 2D geometric transformations on a triangle using Python

PROGRAM:-

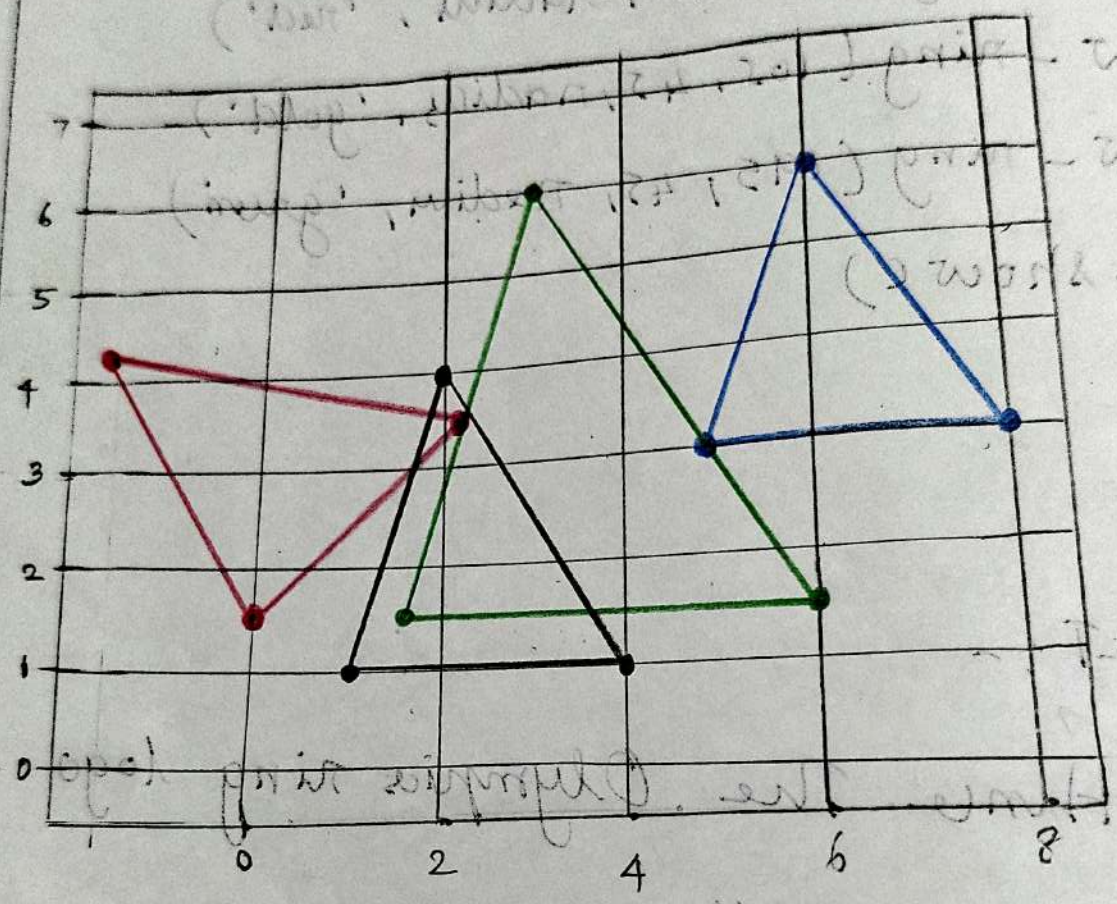
```
import matplotlib.pyplot as plt
import numpy as np

p = np.array([[1,1],[4,1],[2,4],[1,1]])

def draw(p, c):
    plt.plot(p[:,0], p[:,1], 'o-',
             color = c)

draw(p,'black')

t = p + [4,2]
draw(t, 'blue')
```


7
6
5
4
3
2
1
0
1
0
2
4
6
8

```
draw (s, 'green')

a = np.radians (45)
R = np . array ([[np .cos(a), -np .sin (a)],
                 [np. sin(a), np .cos(a)]])

r = p @ R.T
draw(r, 'red')

plt. axis ('equal')
plt.grid()
plt.show()
```

RESULT:-

Hence The implementation of 2D Triangle transformation has been done successfully.

Ex.no.:- 6

Date:- 21/8/26

POLYGON CLIPPING ALGORITHM

AIM:-

To implement polygon clipping Algorithm using python code

PROGRAM:-

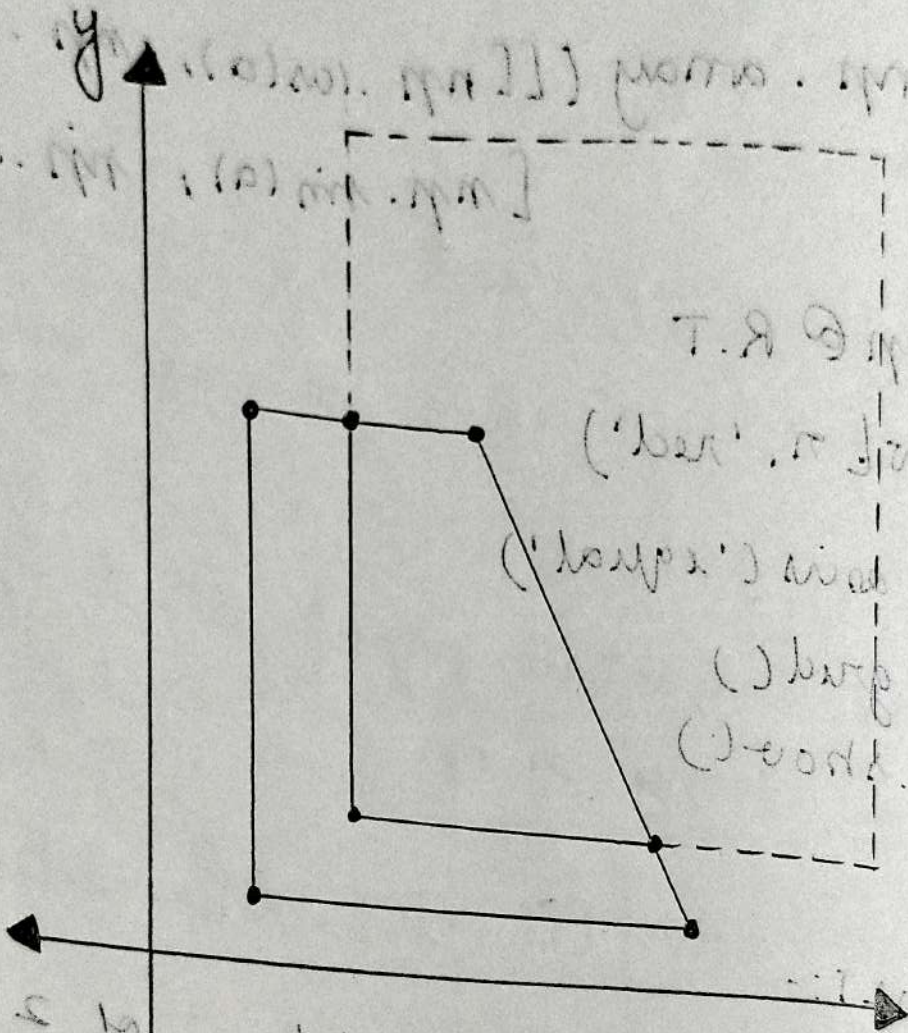
```
import matplotlib.pyplot as plt

p = [(50,50) (150,30) (200,100) (150,170)
     (50,150)]

xmin, ymin = 70,50

def clip(p,c):
    n = []
    for i in range(len(p)):
        A = p[i]; B = p[(i+1)%len(
        ax,ay = A;
```

OUTPUT :



ia = (ax >= xmin if i == 0 else ax <= xmax
if i == 1)

ib = (bx >= xmin if i == 0 else bx <= xmax)

if ia != ib:
if i < 2:
x = xmin if i == 0 else xmax
y = ay + (by - ay) * (x - ax) / (bx - ax)
else:
y = ymin if i == 2 else ymax
x = ax + (bx - ax) * (y - ay) / (by - ay)

```
plt.axis('equal')
plt.grid()
plt.show()
```

RESULT :-

Hence the polygon clipping done successfully

Ex. no. :- 7

Date :- 28/8/26

Implementation of DDA

AIM :-

To implement the DDA (Digital Differential Analyzer) line drawing algorithm.

PROGRAM :-

```
import matplotlib.pyplot as plt
x1, y1 = 2, 3
x2, y2 = 10, 8
dx = x2 - x1
dy = y2 - y1
steps = max(abs(dx), abs(dy))
xinc = dx / steps
yinc = dy / steps
x, y = x1, y1
for i in range(steps + 1):
    x += xinc
    y += yinc
plt.show()
```

RESULT :- Thus, The DDA line drawing was implemented successfully.

Ex. no.:-8
Date:-28/8/26

2D RECTANGLE TRANSFORMATION

AIM:-

To implement 2D rectangle transformations using python.

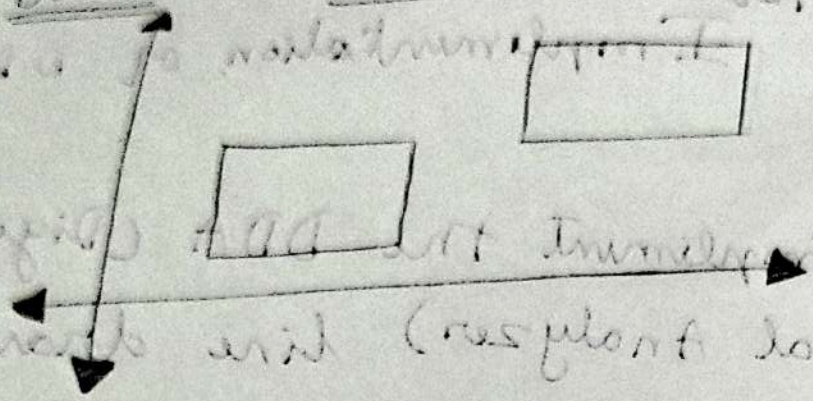
PROGRAM:-

```
import matplotlib.pyplot as plt
import numpy as np.
rect = np.array([[1,1],[4,1],[4,3],[1,3],[1,1]])
translated = rect + [2,2]
scaled = rect * [2,2]
Thita = np.radians(90)
rooted = rect * Thita
plt.legend()
plt.grid()
plt.axis("equal")
plt.show()
```

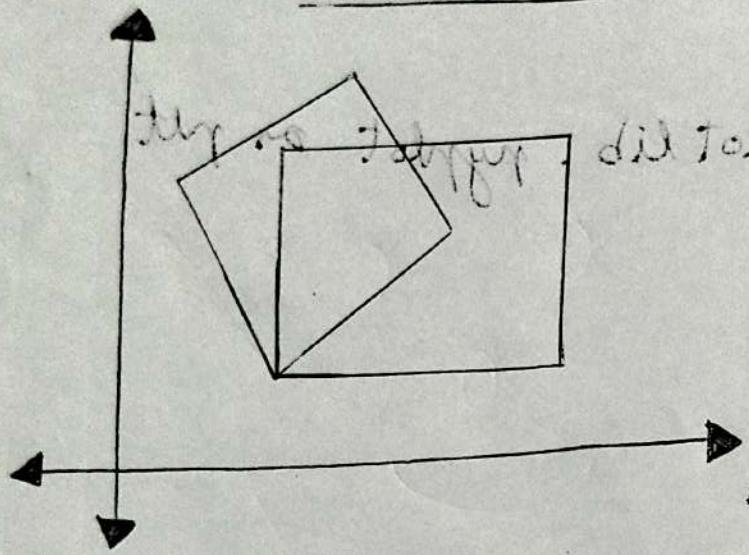
RESULT:-

Thus, 2D rectangle transformations were implemented successfully using python

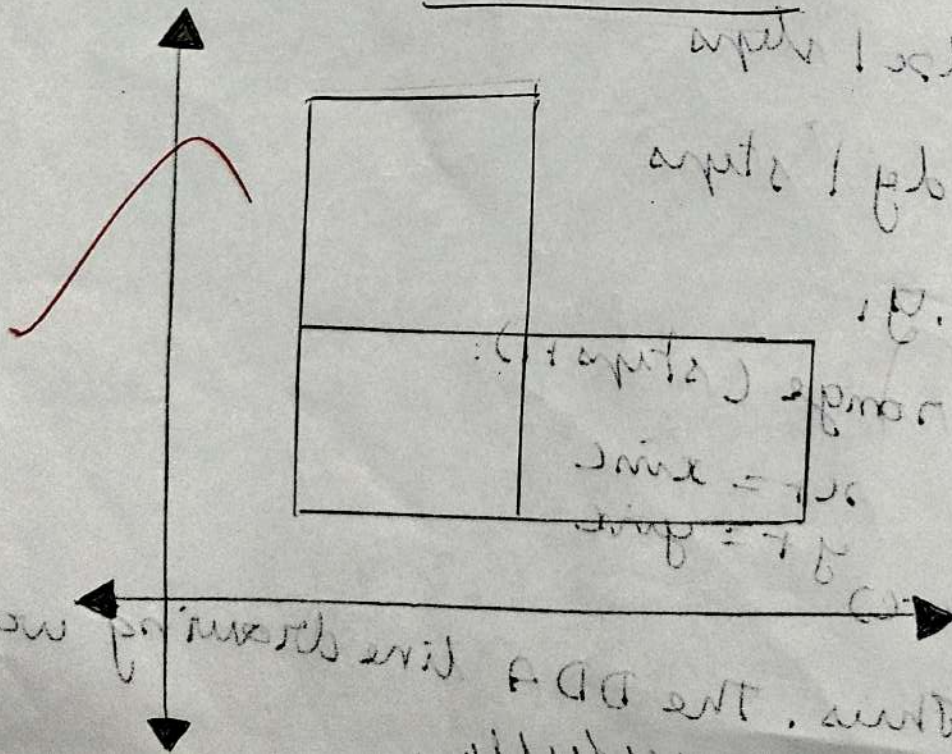
OUTPUT : 2D TRANSLATION



2D ROTATION



2D - SCALING



Ex.no.:-9
Date:-4/9/26

SIMILING FACE

AIM:-

To implement a similing face using ata python based algorithm

PROGRAM:-

```
import matplotlib.pyplot as plt
import numpy as np
def e(cx,cy, rx, ry, c):
    t = np.linspace(0,2*np.pi,200)
    plt.plot(cx+rx*np.cos(t))

e(0,0,5,5)
e(-2,1.5,1.5,.8)
e(2,1.5,5,8)
e(0,0.4,4,.8)
t = np.linspace(np.pi, 2*np.pi,100)
plt.plot()
plt.show()
```

RESULT:-

Hence the similing face has been implemented successfully.

Date: 28/8/20

Exp. No.: 8

3D Rotation Transformation

Aim:

To implement 3D rotation using python program.

Program:

```
import matplotlib.pyplot as plt
import numpy as np
result = np.array([[1,0],[1,1],[1,2],[2,2],[2,1],[2,0]])
translated = result + [2,-2]
scaled = result * [2,-2]
theta = np.random.(90)
rotated = result * theta
plt.legend()
plt.grid()
plt.axis('equal')
plt.show()
```


RESULT:-

Thus, 3D rotation transformation was implemented successfully using python program.

Ex. no. :-10
Date :- 4/9/26

PYTHON MULTIMEDIA INTERFACE

AIM :-

To create a multimedia interface using python with buttons, labels, and audio playback

PROGRAM :-

```
import tkinter as tk
from tkinter import messagebox

root = tk.Tk()
root.title("Multimedia Interface")
tk.Label(root, text = "multimedia player")

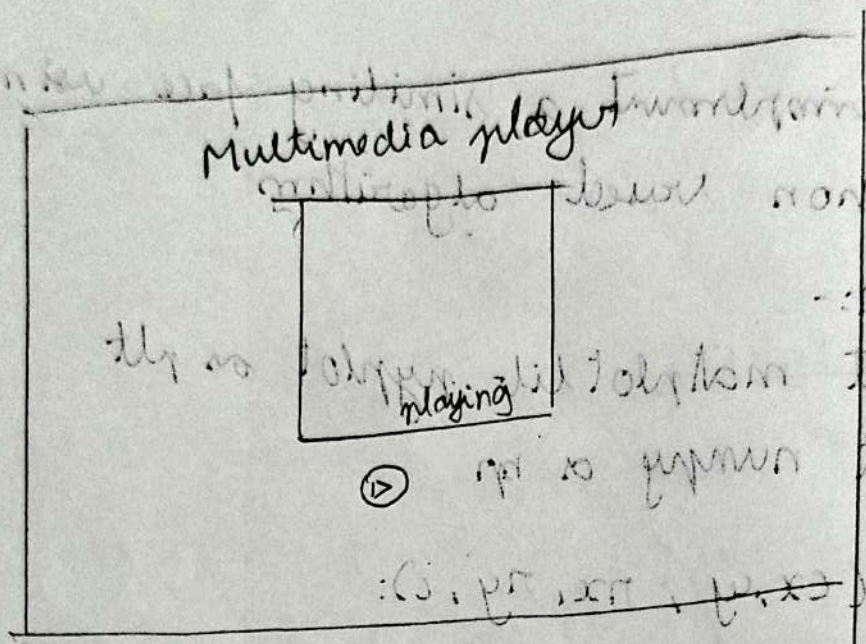
def play():
    messagebox.showinfo("Player", "Playing multimedia")

tk.Button(root, text = "Play", command = play)

root.mainloop()
```

RESULT :-

Thus, a simple multimedia interface was created successfully.


Multimedia player
playing

Ex. no. :- 11
Date :- 8/9/26

IMPLEMENTATION OF SCAN CONVERSION & FLOOD FILL ALGOR

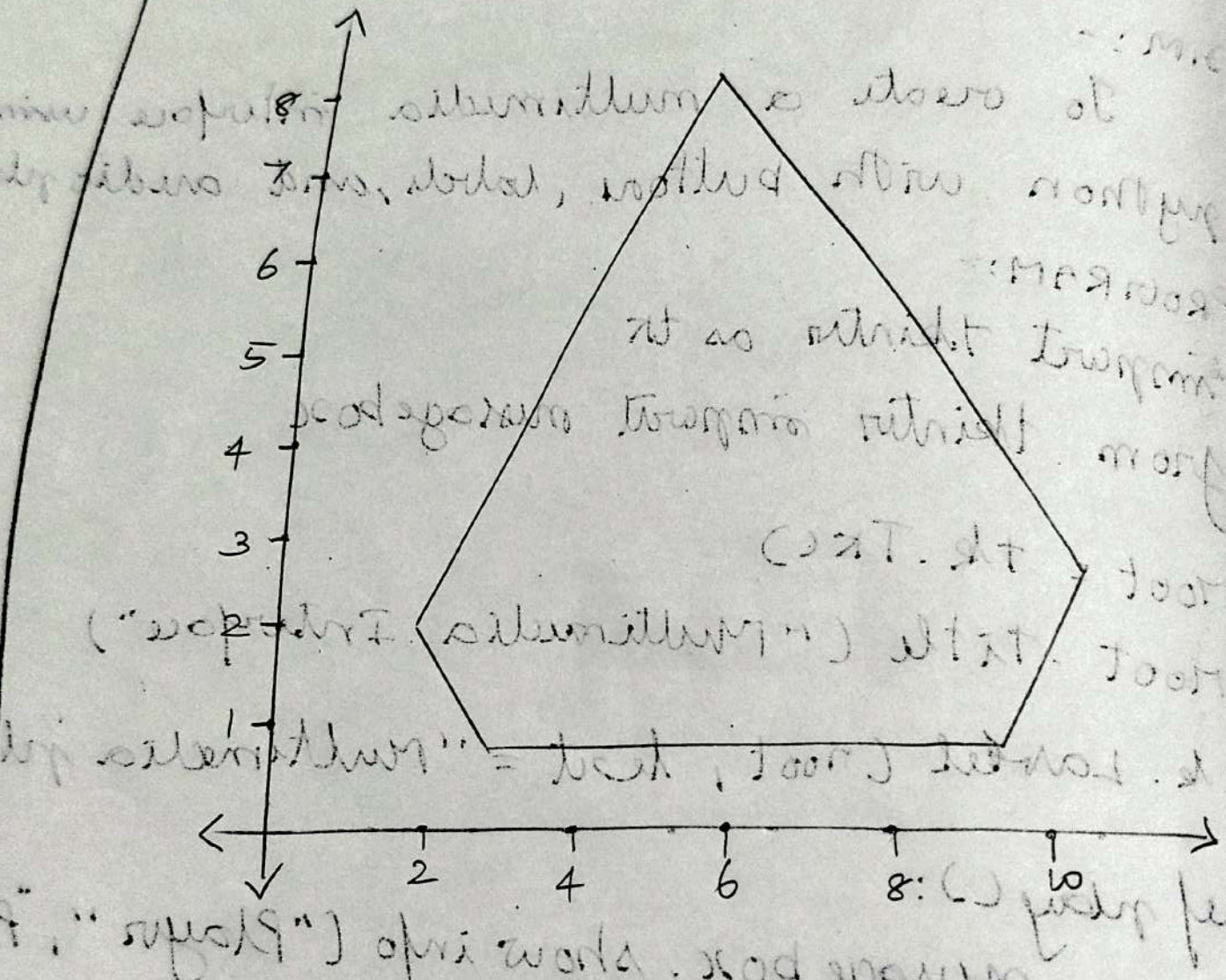
AIM:-

To implement a scan conversion for a polygon and use flood fill algorithm.

PROGRAM:-

```
import matplotlib.pyplot as plt
p = [(2,2),(5,8),(9,3),(7,1),(3,1)]
def scan_fill(p):
    ys = range(min(y for x,y in p),
    for y in ys:
        x = []
        for i in range(len(p)):
            x1,y1 = p[i]
            x.append(x1+(y-y1)*(x2-x1)
        x.sort()
        for i in range(0,len(x),2):
            if i+1 < len(x):
                plt.plot(x[i:i+2],[y
                'b
scan_fill(p)
```

OUTPUT:



```
plt.fill(x,y, color='orange', alpha=0.7)
plt.axis('equal')
plt.title("Scan conversion")
plt.show()
```

RESULT:-

The program for the scan conversion of polygon and use flood fill algorithm is done successfully.

EX.NO :-12

Date :- 11/9/26

MULTIMEDIA MESSAGING RECIEVER APP

AIM :-

To develop a multimedia messaging receiver application using python to receive and display text messages.

PROGRAM :-

```
import tkinter as tk
from tkinter import messagebox
root = tk.Tk()
root.title("Multimedia Msg reciever")
tk.label(root, text = "Message Receiver")
msg.tk.Text(root, height = 5, width = 35)
msg.pack(pady = 10)
def receive():
    messagebox.showinfo("Received Message")
root.mainloop()
```

RESULT :-

Thus, a multimedia messaging receiver application was implemented successfully.

Ex. no:-13

Date:-11/9/26

3D Transformations of a cube

AIM:-

To implement 3D transformations of a cube, including Translation, scaling and rotation using python.

PROGRAM:-

```
import matplotlib.pyplot as plt
import mpl_toolkits.mplot3d import Axes3D
import numpy as np

p = np.array([[0,0,0], [1,0,0], [1,0,0], [0,1,0],
              [0,0,1] [1,0,1], [1,1,1] [0,1,1]])

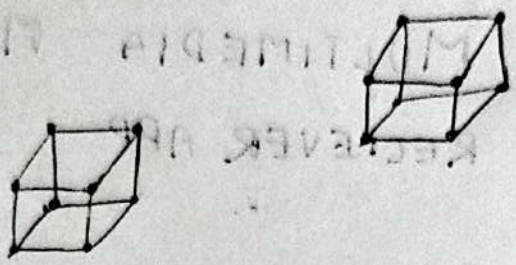
edges = [(0,1) (1,2) (2,3) (3,0) (4,5) (5,6) (6,7) (7,4)]

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
p = p + [2,2,2]
p = p * 2
t = np.radians(45)
n = R @ p
plt.show()
```

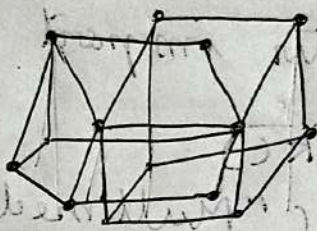
RESULT:- Hence The transformation of a cube is implemented

OUTPUT

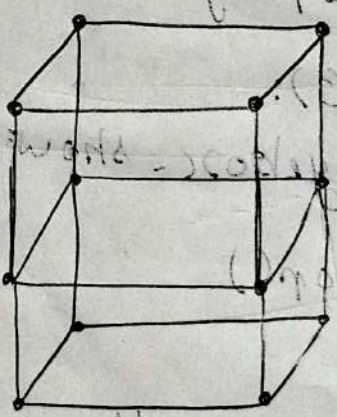
TRANSLATION



ROTATION



SCALING



Ex.NO.:- 14

Date:- 18/9/26

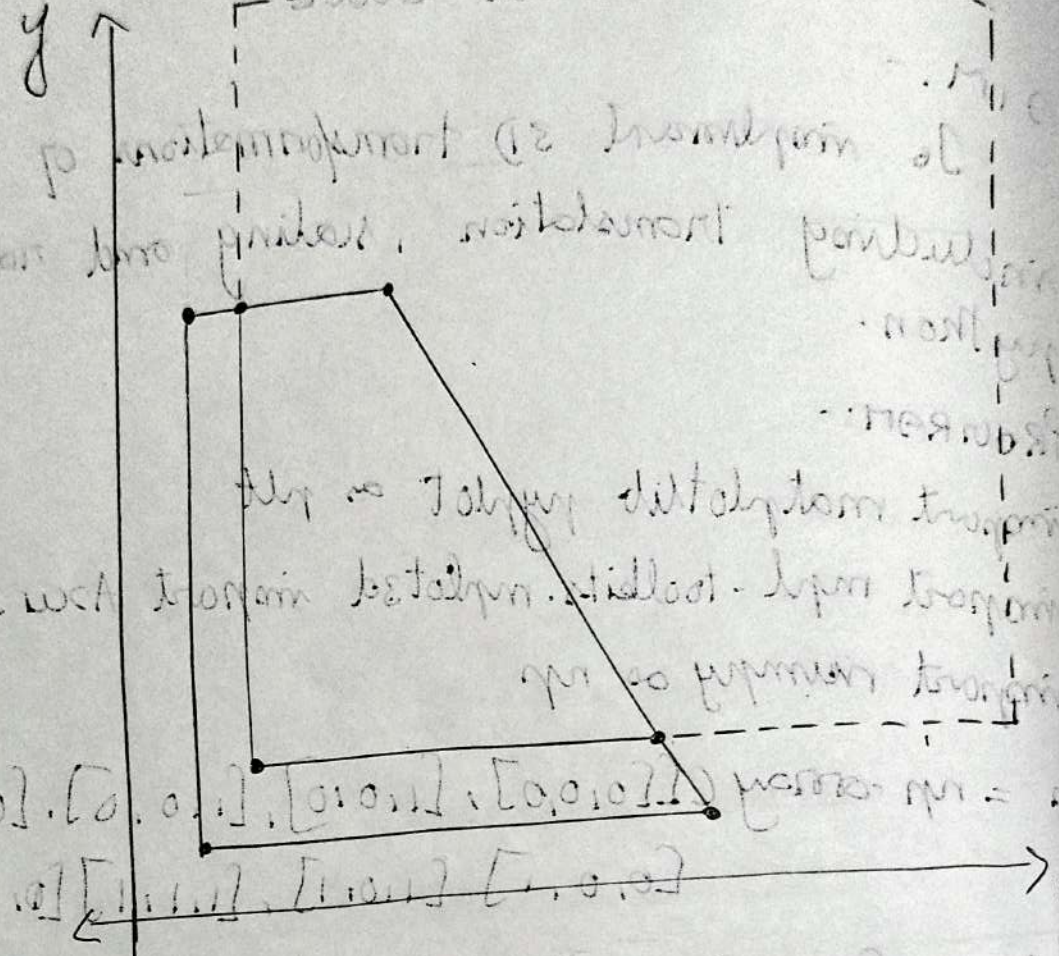
SUTHERLAND - HODGMAN CLIPPING ALGORITHM

AIM:- To implement the algorithm to clip a subject polygon against a rectangular clipping

PROGRAM:-

```
import numpy as np
import matplotlib.pyplot as plt
def inside(p,a,b):
    return np.cross(b-a,p-a)>=0
def intrsct(p1,p2,a,b):
    d1=p2-p1
    d2=b-a
    c=np.cross(a-p1,d2)/np.cross(d1,d2)
    return p2+c*d1
def clip(subject,window):
    output=list(subject)
for i in range(len(window)):
    a,b=window[i],window[i+1]
    inp,output=output,[]
for j in range(len(inp)):
    p1,p2=inp[j-1],inp[j]
```

OUTPUT



```
if in2:
    if not in1:
        output.append(intersect(p1,p2,a,b))

result = clip(subject, window)
plt.plot(subject[i:a]
plt.legend()
plt.show()
```

RESULT:-

The program for sutherland - hodgman polygon is done.

Ex.no.:-5
Date:- 18/9/26

3D TRANSFORMATION FOR CONE

AIM:-

To implement 3D translation, scaling and rotation of a cone.

PROGRAM:-

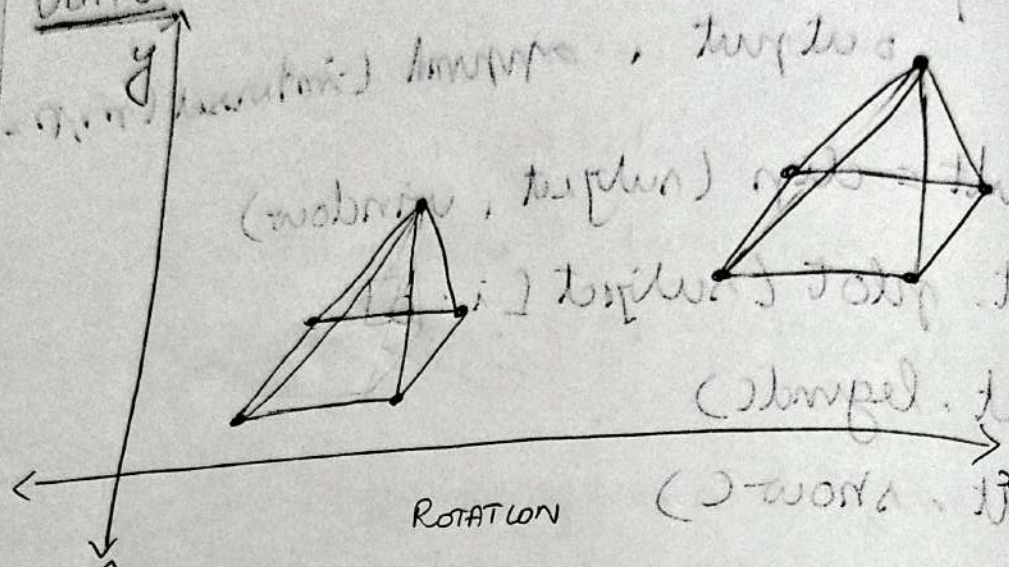
```
import matplotlib.pyplot as plt
import numpy as np
p = np.array([[0,0,0],[0,0,2],[2,2,0]])
edges = [(0,1),(1,2),(2,3),(3,0),(0,4),(1,4)]
p = p+[2,2,1]
p = p*1.5
t = np.radians(45)
p = p @ R.T
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
plt.show()
```

RESULT:-

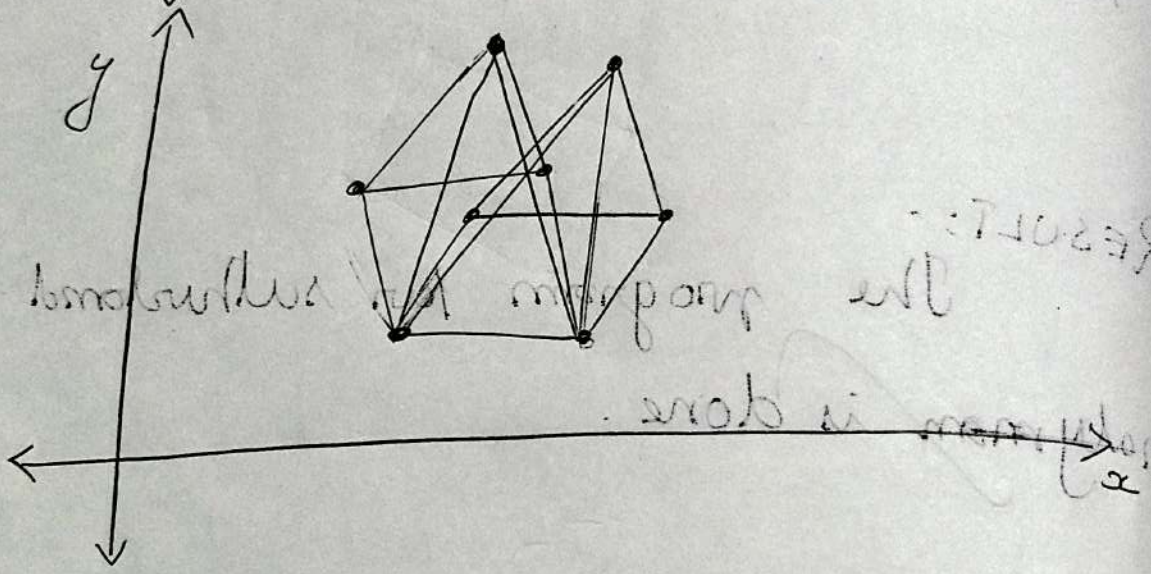
Thus, 3D translation, scaling and rotation of a pyramid was implemented successfully.

TRANSLATION

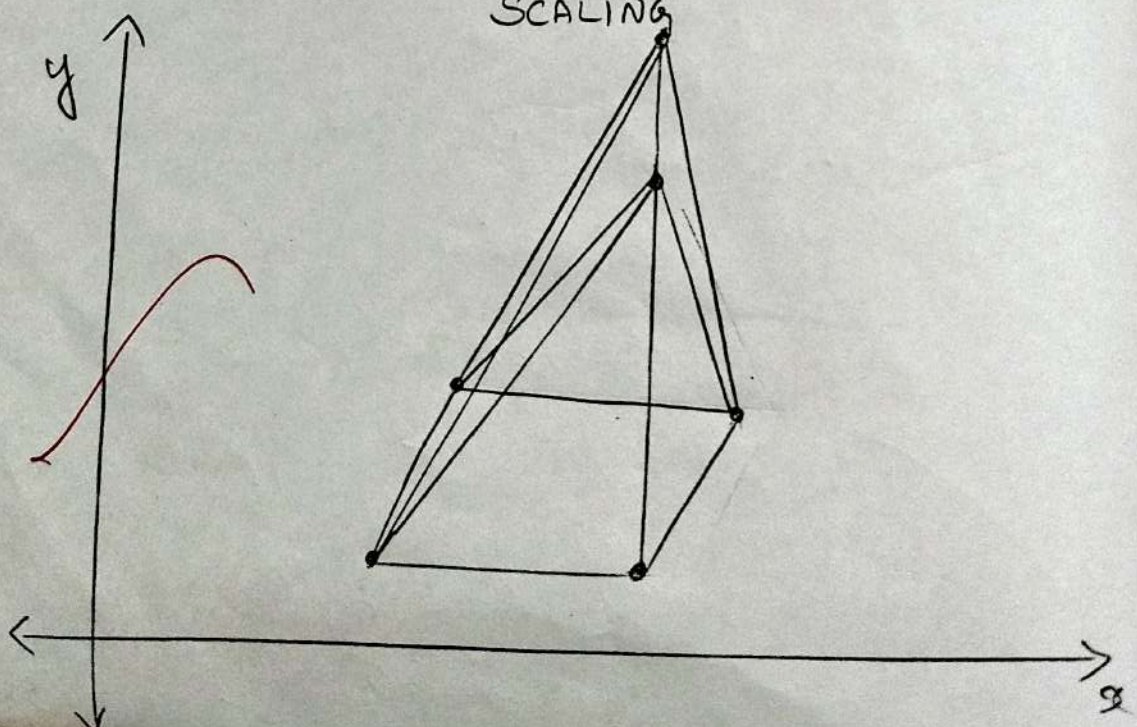
OUTPUT



ROTATION



SCALING



Ex. no. :- 16
Date :- 25/9/26

RENDERING a 3D OBJECT using POLYGON'S SHADING

AIM :- To render a 3D object using polygon shading in Python.

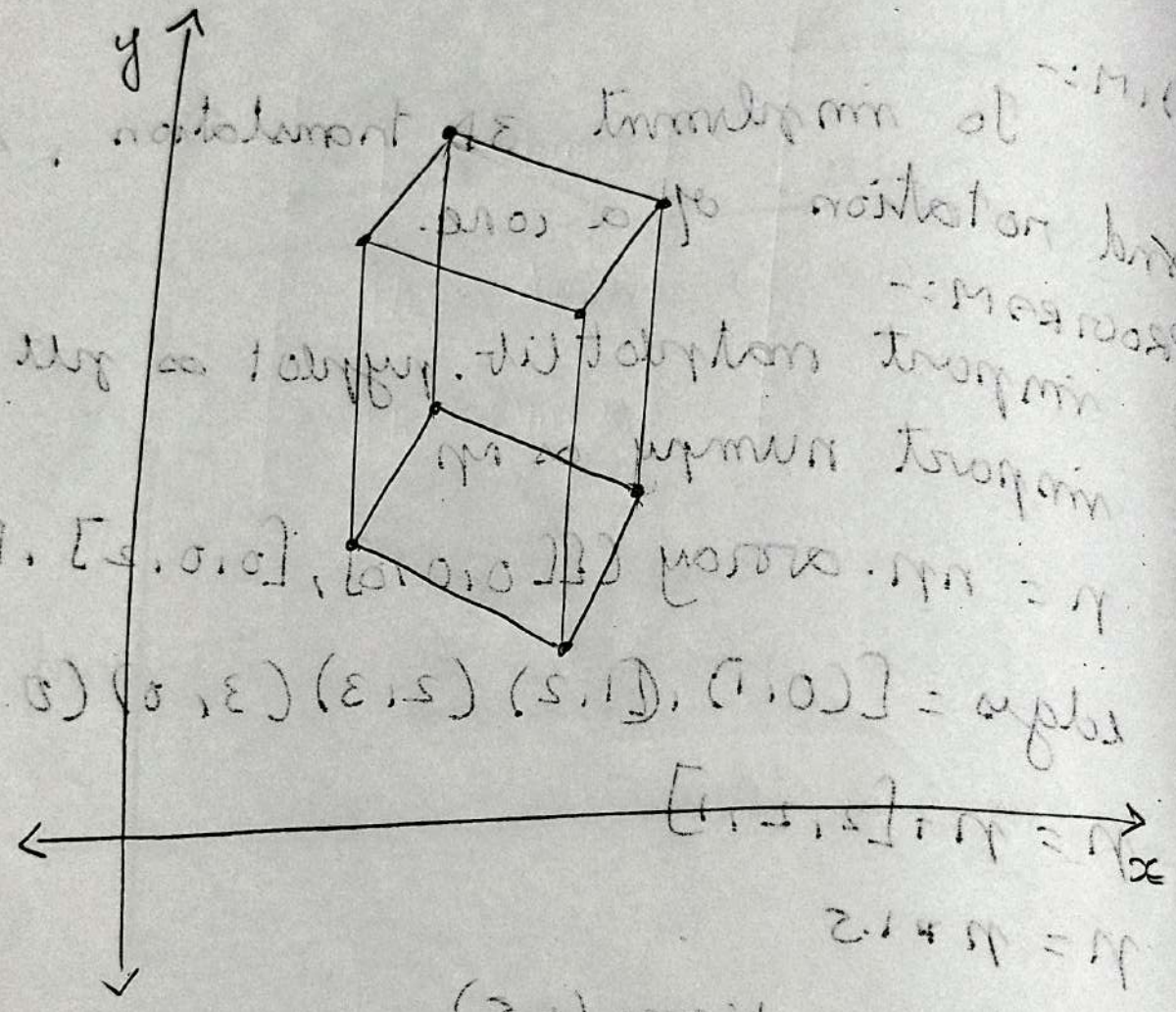
PROGRAM :-

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Poly3DCollection
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
v = [[0,0,0],[1,0,0],[1,1,0],[0,1,0],[0,0,1],[1,0,1],
     [1,1,1],[0,1,1]]
faces = [[v[i] for i in f] for f in
         [[0,1,2,3],[4,5,6,7],[0,1,5,4],[2,3,7,6]]]
cube = Poly3DCollection(faces, facecolors='skyblue',
                        edgecolors='black', alpha=
ax.add_collection3d(cube)
plt.show()
```

RESULT :-

Thus, a 3D object was rendered successfully using polygon shading in python

OUTPUT



Ex.no.:-17
Date :- 25/9/26

MULTIMEDIA APP INTEGRATES IMAGES, VIDEOS AND AUDIOS

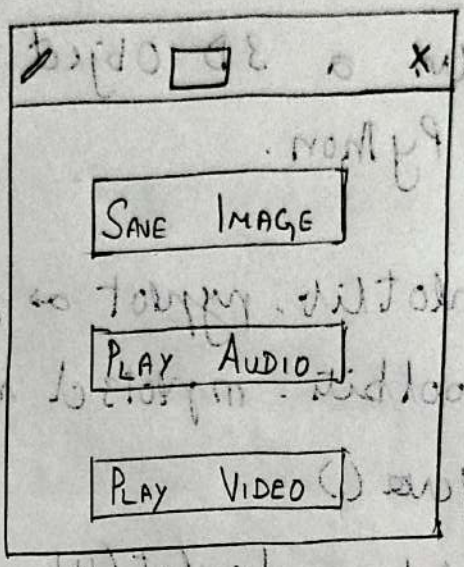
AIM:-

To develop a multimedia application using python that integrates images, videos & audio files

PROGRAM:-

```
import os
root = tk.Tk()
root.title("Multimedia APP")
def open_image():
    f = filedialog.askopenfilename(
        filetypes=[("Images", "*.png")])
    if f:
        img = Image.open(f)
        pic = ImageTk.PhotoImage(img)
        display.image = pic
def open_audio():
    f = filedialog.askopenfilename(filetypes=[("Audio
                                   "*.mp3")])
    if f:
        os.startfile(f)
```

OUTPUT



```
tk.Button(root, text = "Open Image",
          command = open_image).pack()
tk.Button(root, text = "Play Video",
          command = open_video).pack()
tk.Button(root, text = "Play Audio",
          command = open_audio).pack()
root.mainloop()
```

RESULT:-

Thus a python multimedia application integrating images, audios and videos was implemented successfully.

Ex. no :- 18
Date :- 21/10/26

CAPTURE VIDEO AND AUDIO AND DISPLAY THE MULTIMEDIA

AIM:-

To capture video from a webcam and audio from a microphone and display the multimedia output.

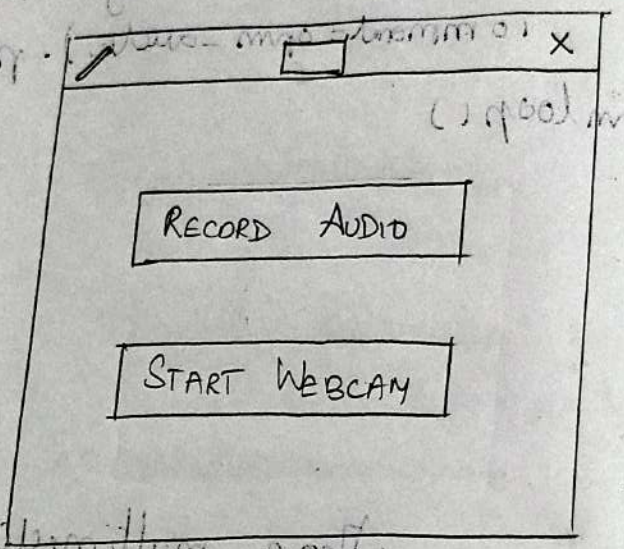
PROGRAM:-

```
import cv2
import sounddevice as sd
from PIL import Image, ImageTk
import tkinter as tk

root = tk.Tk()
root.title("Audio video capture")
cam = cv2.VideoCapture(0)
audio = none
recording = False

label.pack()
def show_video():
    ret, frame = cam.read()
    if ret:
```

OUTPUT:



```
    from = cv2.cvtColor(frame)
    label.config(img)
    label.image = img
def record_audio():
    global audio, recording
    if not recording:
        recording = True
        audio = sd.rec(44100 * 5)
        root.after(5000, stop_audio)
show_video()
root.mainloop()
cam.release()
```

RESULT :-

Thus, live video was captured and displayed in a Tkinter window

Ex. NO :- 19
Date :- 2/10/26

OBJECT USING DDA ALGORITHM

AIM:-
To draw a 2D object using the DDA line drawing algorithm

PROGRAM:-

```
import matplotlib.pyplot as plt

def dda(x1,y1,x2,y2):
    dx,dy = x2-x1,y2-y1
    steps = max(abs(dx),abs(dy))
    xinc,yinc = dx/steps,dy/steps
    x,y = x1,y1
    for i in range(steps+1):
        plt.plot(round(x),round(y),'ro
        x+=xinc
        y+=yinc

dda(2,2,8,2)
dda(8,2,8,6)
plt.show()
```

RESULT:-
Thus a object is drawn using DDA

OUTPUT

Resulting 2D Object (Rectangle)

